Actividad: prepara una mochila con un algoritmo genético

Enunciado

Vas a pasar un día en la montaña y tienes una mochila que puede cargar **como máximo 10 kg**. Dispones de varios objetos, cada uno con un peso y una puntuación de utilidad.

Tu objetivo es programar en **Python** un algoritmo genético que elija qué objetos llevar para conseguir la mayor utilidad posible sin superar el peso máximo. Solo puedes llevar una unidad de cada objeto.

Objeto	**Peso (kg)**	**Utilidad**
Agua	3	10
Comida	2	9
Abrigo	2	7
Botiquín	1	8
Linterna	1	5
Tienda de campaña	5	6
Cámara	2	4
Libro	1	2

Funcionamiento del algoritmo

1. **Representación de las soluciones.** Representa cada solución con una lista de ocho bits, siguiendo el orden de los objetos de la tabla. Un `1` indica que llevas el objeto y un `0` que lo dejas. Por ejemplo, `11000000` representa llevar agua y comida.

2. **Población inicial.** Genera una población de 20 soluciones aleatorias.

3. **Evaluación.** La puntuación de una solución será la suma de las utilidades de los objetos elegidos. Si el peso total supera los 10 kg, su puntuación será 0.

4. **Selección mediante torneos.** Para seleccionar un padre, elige dos individuos distintos al azar y conserva el de mayor puntuación. En caso de empate, elige uno al azar. Repite el procedimiento para seleccionar al segundo padre.

5. **Cruce.** Elige al azar un punto de corte entre las posiciones 1 y 7. Intercambia las partes finales de los dos padres para crear dos hijos.

6. **Mutación.** Cada bit de cada hijo tendrá un 5 % de probabilidad de cambiar su valor: de 0 a 1 o de 1 a 0.

7. **Nueva generación.** Conserva una copia de la mejor solución de la generación anterior sin modificarla. Añade hijos obtenidos mediante selección, cruce y mutación hasta completar una población de 20 individuos. Si solo queda un hueco, añade únicamente uno de los dos hijos.

8. **Finalización.** Repite el proceso durante 100 generaciones y muestra la mejor solución válida encontrada. Puedes utilizar la mochila vacía como solución válida inicial.

Organización obligatoria del código

El programa debe estar organizado en los siguientes cinco archivos:

`fuse.py` Archivo principal del programa. Define los datos de los objetos y los parámetros, genera la población inicial, evalúa las soluciones, ejecuta las 100 generaciones y muestra el resultado. Debe utilizar las funciones de los otros módulos.

`torneo.py` Implementa un torneo entre dos individuos elegidos al azar y devuelve el ganador según su puntuación.

`seleccion.py` Selecciona los dos padres necesarios para un cruce, utilizando la función definida en `torneo.py`.

`cruce.py` Implementa el cruce de un punto entre dos padres y devuelve los dos hijos.

`mutacion.py` Implementa la mutación de un individuo, aplicando una probabilidad del 5 % a cada bit.

El programa debe poder ejecutarse con el siguiente comando:

```
python fuse.py
```

Resultados que debe mostrar

- Los objetos elegidos.
- El peso total de la mochila.
- La utilidad total de la solución.

No es necesario encontrar siempre la solución óptima, pero la solución presentada debe respetar el límite de peso.

Entrega

Entrega los cinco archivos de Python y una breve respuesta a la siguiente pregunta:

> Si ejecutas el programa cinco veces, ¿obtienes siempre el mismo resultado? ¿Por qué?